

HW 4 Computer Vision

This assignment is designed to assess your understanding of key concepts in Stereo Vision detection.

Submission Guidelines:

- Notebook Submission: Complete your solutions within this notebook.
- PDF Submission: Export your completed notebook as a PDF and submit it alongside the notebook.

Important Notes:

- Submissions without a PDF file will incur a 10% penalty on the total grade.
- Ensure that your code runs without errors and that all visual outputs are included in the PDF.
- Write clear and concise explanations for your code and make sure to add comments in each part of your code.
- Code with no comments will lose 10% of the question mark. Properly format your code and comments to enhance readability.

Part 1 MCQs (15 marks)

Q1. What is the primary function of the pinhole camera model in stereo vision?

- a) Increase light exposure
- b) Project 3D scene points onto a 2D image plane
- c) Eliminate lens distortion
- d) Calibrate color mapping

Ans: B

Q2. What is the disadvantage of having a very small pinhole?

- a) It causes blurring
- b) It overexposes the image

- c) It lets in less light
- d) It removes depth information

Ans: C

Q3. Which matrix encodes the rotation and translation between two views?

- a) Homography matrix
- b) Fundamental matrix
- c) Essential matrix
- d) Intrinsic matrix

Ans: C

Q4. What does the Essential Matrix map?

- a) Points to points
- b) Lines to points
- c) Points to lines
- d) Points to planes

Ans: C

Q5. The fundamental matrix differs from the essential matrix in that it accounts for:

- a) Lens distortion
- b) Intrinsic parameters
- c) Extrinsic parameters
- d) Homogeneous coordinates

Ans: B

Q6. What is the minimum number of point correspondences needed to compute the essential matrix using the 8-point algorithm?

- a) 4
- b) 6

- c) 8
- d) 12

Ans: C

Q7. What is the goal of image rectification?

- a) Convert image to grayscale
- b) Make images identical
- c) Align epipolar lines horizontally
- d) Blur the background

Ans: C

Q8. Which of the following is used for computing depth in stereo vision?

- a) Feature maps
- b) Disparity maps
- c) Color histograms
- d) Optical flow

Ans: B

Q9. Disparity is inversely proportional to:

- a) Brightness
- b) Camera calibration
- c) Depth
- d) Focal length

Ans: C

Q10. Which OpenCV function helps estimate the 3D structure from matched points?

- a) `cv2.stereoRectify()`
- b) `cv2.findContours()`

- c) `cv2.StereoBM_create()`
- d) `cv2.triangulatePoints()`

Ans: D

Q11. What does the projection matrix of a camera consist of?

- a) Only intrinsic matrix
- b) Intrinsic and extrinsic parameters
- c) Only rotation matrix
- d) Only translation vector

Ans: B

Q12. Dense reconstruction differs from sparse reconstruction because:

- a) It uses a different matrix
- b) It reconstructs only background pixels
- c) It computes depth for all pixels
- d) It skips rectification

Ans: C

Q13. What type of points are typically used in sparse 3D reconstruction?

- a) Random pixels
- b) SIFT keypoints
- c) Disparity vectors
- d) Color-matched pixels

Ans: B

Q14. The formula $F = K_2^T (-T) E K_1^{-1}$ computes:

- a) Camera matrix
- b) Homography

- c) Fundamental matrix
- d) Essential matrix

Ans: C

Q15. What is the main idea behind block matching in depth map estimation?

- a) Apply a filter to the disparity
- b) Match color histograms
- c) Match pixel blocks along the x-axis
- d) Compute histograms over the entire image

Ans: C

Part 2 Coding Tasks (10 marks each)

Q1 (10 marks)

Objective: Estimate the disparity map from a stereo image pair using block matching.

You are provided with a stereo image pair (im0.png, im1.png) from the artroom1 scene.

You will use the built-in `cv2.StereoBM_create()` method, which internally uses the Sum of Absolute Differences (SAD):

$$\text{SAD}(d) = \sum_{i=-k}^k \sum_{j=-k}^k |I_L(x+i, y+j) - I_R(x-d+i, y+j)|$$

Instructions

1. Load left and right images in grayscale (1 mark)
2. Correct use of StereoBM_create() with valid parameters (2 marks)
3. Compute disparity with .compute() (2 marks)
4. Normalize and visualize the disparity (3 marks)
5. Clean, readable, and commented code (2 marks)

```
In [ ]: import cv2
import numpy as np
import matplotlib.pyplot as plt

#Load the stereo images in grayscale
img1 = cv2.imread('./HW4 inputs/im0.png', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('./HW4 inputs/im1.png', cv2.IMREAD_GRAYSCALE)

sift = cv2.SIFT_create()
# find the keypoints and descriptors with SIFT
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# Create BFMatcher (Brute Force Matcher)
```

```

bf = cv2.BFMatcher()
matches = bf.knnMatch(des1,des2, k=2)

pts1 = []
pts2 = []
# ratio test as per Lowe's paper
for i,(m,n) in enumerate(matches):
    if m.distance < 0.8*n.distance:
        pts1.append(kp1[m.queryIdx].pt)
        pts2.append(kp2[m.trainIdx].pt)

# Finding Fundamental matrix
pts1_ = np.array(pts1)
pts2_ = np.array(pts2)
F, mask = cv2.findFundamentalMat(pts1_, pts2_, cv2.RANSAC)

# We select only inlier points
pts1_ = pts1_[mask.ravel()==1]
pts2_ = pts2_[mask.ravel()==1]

h,w = img1.shape
# find the two homographies
_, H1, H2 = cv2.stereoRectifyUncalibrated(pts1_, pts2_, F, (w,h))
# Applying the two homographies to the two images to rectify them
rectified_img1 = cv2.warpPerspective(img1, H1, (w,h), flags=cv2.INTER_LINEAR)
rectified_img2 = cv2.warpPerspective(img2, H2, (w,h), flags=cv2.INTER_LINEAR)

# Plots
plt.subplot(121),plt.imshow(rectified_img1, cmap='gray'), plt.axis('off')
plt.subplot(122),plt.imshow(rectified_img2, cmap='gray'), plt.axis('off')
plt.show()

# Create a StereoBM object with valid parameters
stereo = cv2.StereoBM_create(numDisparities=16, blockSize=21)

# Compute the disparity map
disparity = stereo.compute(rectified_img1,rectified_img2)

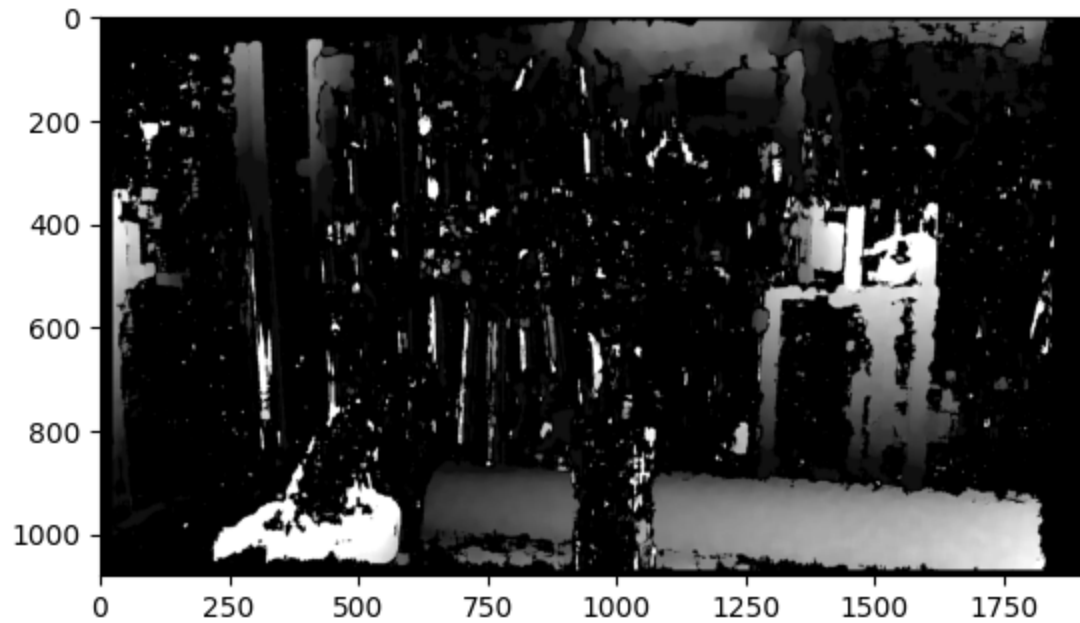
# Normalize the disparity map for visualization
disparity_normalized = cv2.normalize(disparity, None, alpha=0, beta=255, norm_type=cv2.NORM_MINMAX, dtype=cv2.CV_8U)

```

```
# Visualize the disparity map  
plt.imshow(disparity_normalized, 'gray')
```



Out[]: <matplotlib.image.AxesImage at 0x27218c71b50>



Q2 (10 marks)

Compute the depth map from a disparity map using camera calibration parameters.

You are given:

A disparity map (disp0.pfm)

The calibration file (calib.txt) containing the focal length and baseline

Use the formula:

$$\text{Depth}(x, y) = \frac{f \cdot B}{d(x, y)}$$

Instructions:

1. Load disparity map (1 mark)
2. Parse and extract focal length + baseline (4 marks)
3. Apply depth formula with safe division (3 marks)
4. Visualize or display depth map (2 marks)

```
In [25]: import cv2
import numpy as np

# Load the disparity map
import matplotlib.pyplot as plt

disparity = cv2.imread('./HW4 inputs/dis0.pfm', cv2.IMREAD_UNCHANGED)

# Parse the calibration file to extract focal length and baseline
with open('./HW4 inputs/calib.txt', 'r') as f:
    lines = f.readlines()
    for line in lines:
        if 'baseline' in line:
            baseline = float(line.split('=')[1].strip())
# Extract focal length from the cam0 matrix
    for line in lines:
        if 'cam0' in line:
            cam0_matrix = line.split('=')[1].strip('[]\n').split(';')[0]
            focal_length = float(cam0_matrix.split()[0])
            break

# Compute the depth map using the formula Depth(x, y) = (f * B) / d(x, y)
# Avoid division by zero by replacing zero disparities with a small value
epsilon = 1e-6
disparity[disparity == 0] = epsilon
```

```
depth = (focal_length * baseline) / disparity
```

```
# Visualize the depth map in grayscale  
plt.imshow(depth, cmap='gray')
```

Out[25]: <matplotlib.image.AxesImage at 0x27218d88e10>

